

Guião 09 – Mensageria e Desacoplamento com RabbitMQ

Infraestruturas de Computação na Nuvem, Aula 09

1 Objetivos

- Implantar um broker RabbitMQ no cluster Kubernetes, com armazenamento e Service próprios.
- Implementar um producer e um consumer para uma fila de trabalho (*work queue*), correndo como Pods.
- Testar o padrão publish/subscribe com um exchange *fanout* e múltiplos consumers.
- Observar o efeito do desacoplamento ao parar temporariamente um consumer, e a acumulação de mensagens na fila.
- Escalar o número de consumers e verificar a distribuição de trabalho.
- Observar a redundância ativa-ativa e o efeito de *anti-affinity* entre nós.

2 Pré-requisitos e Material

- O cluster Kubernetes de dois nós (Guião 05), com `kubectl` configurado.
- Acesso ao Proxmox, para o teste final de falha de nó.
- Conhecimentos de Deployment, Service e escalamento das Aulas 05 e 08.

Alternativa mais leve: se o cluster não estiver disponível, todos os exercícios podem ser feitos com Docker dentro da VM `icn-lab` – arrancando o RabbitMQ com `docker run -d -p 5672:5672 -p 15672:15672 rabbitmq:3-management` e correndo os scripts Python diretamente na VM. Nesse caso, saltam-se as Partes 1 e 5.

3 Introdução

Num sistema desacoplado, os serviços comunicam através de um intermediário – o *broker* de mensagens – em vez de se chamarem diretamente. O RabbitMQ implementa o protocolo AMQP, organizando a comunicação em torno de *exchanges* (onde os produtores publicam) e *filas* (de onde os consumidores retiram).

Neste guião implantamos o broker no cluster Kubernetes e exploramos dois padrões: a **work queue**, em que cada mensagem é processada por um único consumer de entre vários, e o **publish/subscribe**, em que cada mensagem chega a todos os subscritores. O objetivo central é observar concretamente a propriedade que torna esta arquitetura resiliente: *o produtor não é afetado quando o consumidor falha*.

4 Parte 1 – Implantar o RabbitMQ no cluster

1. **Criar o manifesto rabbitmq.yaml.** Note que o broker guarda estado, pelo que usamos um Service dedicado e credenciais explícitas:

```
apiVersion: v1
kind: Secret
metadata:
  name: rabbitmq-cred
type: Opaque
stringData:
  RABBITMQ_DEFAULT_USER: "aluno"
  RABBITMQ_DEFAULT_PASS: "aulaicn2026"
---
apiVersion: apps/v1
kind: Deployment
metadata:
  name: rabbitmq
spec:
  replicas: 1
  selector:
    matchLabels:
      app: rabbitmq
  template:
    metadata:
      labels:
        app: rabbitmq
    spec:
      containers:
        - name: rabbitmq
          image: rabbitmq:3-management
          envFrom:
            - secretRef:
                name: rabbitmq-cred
          ports:
            - containerPort: 5672
              name: amqp
            - containerPort: 15672
              name: management
          readinessProbe:
            exec:
              command: ["rabbitmq-diagnostics", "-q", "ping"]
            initialDelaySeconds: 20
            periodSeconds: 10
---
apiVersion: v1
kind: Service
metadata:
  name: rabbitmq
spec:
  selector:
    app: rabbitmq
  ports:
    - name: amqp
      port: 5672
      targetPort: 5672
    - name: management
      port: 15672
```

```
targetPort: 15672
nodePort: 31672
type: NodePort
```

Usamos um **Secret** para as credenciais, aplicando o que foi visto na Aula 05. Note que, por omissão, um Secret é apenas codificado em base64 – não cifrado.

2. Aplicar e aguardar que fique pronto:

```
$ kubectl apply -f rabbitmq.yaml
$ kubectl get pods -l app=rabbitmq --watch
```

O arranque demora cerca de meio minuto. Espere pelo estado **Running** com 1/1 **READY**.

3. Aceder à interface de gestão em `http://<IP-de-um-no>:31672`, com as credenciais definidas no Secret. Explore os separadores *Overview*, *Exchanges* e *Queues*.

4. Confirmar a resolução do Service por DNS a partir de dentro do cluster:

```
$ kubectl run dns-teste --rm -it --image=busybox:1.36 --restart=Never -- \
nslookup rabbitmq
```

Os Pods poderão alcançar o broker simplesmente pelo nome `rabbitmq`.

5 Parte 2 – Work queue

5. Criar o producer `send_task.py`. Repare nas duas marcas de durabilidade: a fila é `durable` e as mensagens têm `delivery_mode=2`:

```
import pika, sys, os

cred = pika.PlainCredentials(
    os.environ.get('RABBIT_USER', 'aluno'),
    os.environ.get('RABBIT_PASS', 'aula1cn2026'))
params = pika.ConnectionParameters(
    host=os.environ.get('RABBIT_HOST', 'rabbitmq'),
    credentials=cred)

connection = pika.BlockingConnection(params)
channel = connection.channel()
channel.queue_declare(queue='tarefas', durable=True)

mensagem = ' '.join(sys.argv[1:]) or "tarefa generica"
channel.basic_publish(
    exchange='',
    routing_key='tarefas',
    body=mensagem,
    properties=pika.BasicProperties(delivery_mode=2)
)
print(f"Enviado: {mensagem}")
connection.close()
```

6. Criar o consumer `worker.py`. Note o `basic_ack` explícito e o `prefetch_count=1`:

```
import pika, time, os, socket

cred = pika.PlainCredentials(
    os.environ.get('RABBIT_USER', 'aluno'),
```

```

    os.environ.get('RABBIT_PASS', 'aula1cn2026'))
params = pika.ConnectionParameters(
    host=os.environ.get('RABBIT_HOST', 'rabbitmq'),
    credentials=cred)

connection = pika.BlockingConnection(params)
channel = connection.channel()
channel.queue_declare(queue='tarefas', durable=True)

nome = socket.gethostname()

def callback(ch, method, properties, body):
    print(f"[{nome}] Recebido: {body.decode()}", flush=True)
    time.sleep(2)
    print(f"[{nome}] Concluido", flush=True)
    ch.basic_ack(delivery_tag=method.delivery_tag)

channel.basic_qos(prefetch_count=1)
channel.basic_consume(queue='tarefas', on_message_callback=callback)
print(f"[{nome}] A espera de tarefas.", flush=True)
channel.start_consuming()

```

7. **Correr os workers como Pods no cluster.** Para evitar construir uma imagem, usamos uma imagem Python base e injetamos o script através de um ConfigMap:

```
$ kubectl create configmap worker-code --from-file=worker.py
```

Crie worker.yaml:

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: worker
spec:
  replicas: 2
  selector:
    matchLabels:
      app: worker
  template:
    metadata:
      labels:
        app: worker
    spec:
      containers:
        - name: worker
          image: python:3.12-slim
          command: ["sh", "-c"]
          args:
            - "pip install --no-cache-dir pika && python /code/worker.py"
          envFrom:
            - secretRef:
                name: rabbitmq-cred
          env:
            - name: RABBIT_USER
              value: "aluno"
            - name: RABBIT_PASS
              value: "aula1cn2026"
            - name: RABBIT_HOST

```

```

        value: "rabbitmq"
    volumeMounts:
      - name: code
        mountPath: /code
    volumes:
      - name: code
        configMap:
          name: worker-code

```

```

$ kubectl apply -f worker.yaml
$ kubectl get pods -l app=worker

```

8. **Acompanhar os logs dos workers** num terminal:

```
$ kubectl logs -f -l app=worker --prefix --max-log-requests=10
```

9. **Enviar tarefas** a partir de um Pod temporário, noutro terminal:

```

$ kubectl run producer --rm -it --image=python:3.12-slim --restart=Never \
  --env="RABBIT_HOST=rabbitmq" --env="RABBIT_USER=aluno" \
  --env="RABBIT_PASS=aula1cn2026" -- sh
# pip install pika
# (colar aqui o conteudo de send_task.py num ficheiro e correr varias vezes)

```

Alternativa mais simples: publicar diretamente pela interface de gestão web (separador *Queues* → *tarefas* → *Publish message*), enviando várias mensagens seguidas.

10. **Observar a distribuição.** Nos logs, confirme que as tarefas são repartidas pelos workers ativos, e não processadas todas pelo mesmo. Registe a repartição observada.
11. **Escalar os consumers** e repetir:

```
$ kubectl scale deployment worker --replicas=4
```

Envie mais tarefas e compare o tempo total de processamento com o obtido com 2 workers.

6 Parte 3 – Publish/subscribe com fanout

12. **Criar o publisher** `publish_event.py`:

```

import pika, sys, os

cred = pika.PlainCredentials(
    os.environ.get('RABBIT_USER', 'aluno'),
    os.environ.get('RABBIT_PASS', 'aula1cn2026'))
params = pika.ConnectionParameters(
    host=os.environ.get('RABBIT_HOST', 'rabbitmq'),
    credentials=cred)

connection = pika.BlockingConnection(params)
channel = connection.channel()
channel.exchange_declare(exchange='eventos', exchange_type='fanout')

mensagem = ' '.join(sys.argv[1:]) or "evento generico"
channel.basic_publish(exchange='eventos', routing_key='', body=mensagem)
print(f"Evento publicado: {mensagem}")
connection.close()

```

13. **Criar o subscriber** `subscribe_event.py`. Cada instância cria a sua própria fila temporária e exclusiva, ligada ao exchange:

```
import pika, os, socket

cred = pika.PlainCredentials(
    os.environ.get('RABBIT_USER', 'aluno'),
    os.environ.get('RABBIT_PASS', 'aulaicn2026'))
params = pika.ConnectionParameters(
    host=os.environ.get('RABBIT_HOST', 'rabbitmq'),
    credentials=cred)

connection = pika.BlockingConnection(params)
channel = connection.channel()
channel.exchange_declare(exchange='eventos', exchange_type='fanout')

result = channel.queue_declare(queue='', exclusive=True)
queue_name = result.method.queue
channel.queue_bind(exchange='eventos', queue=queue_name)

nome = socket.gethostname()

def callback(ch, method, properties, body):
    print(f"[{nome}] recebeu: {body.decode()}", flush=True)

channel.basic_consume(queue=queue_name, on_message_callback=callback, auto_ack=True)
print(f"[{nome}] A espera de eventos.", flush=True)
channel.start_consuming()
```

14. **Implantar dois ou mais subscribers** (adapte o `worker.yaml`, mudando o nome, o `ConfigMap` e o script a executar) e publique um evento.
15. **Confirmar a diferença de comportamento:** na work queue, cada mensagem foi processada por *um* worker; aqui, o mesmo evento chega a *todos* os subscribers. Verifique também, na interface de gestão, que cada subscriber tem a sua própria fila ligada ao exchange `eventos`.

7 Parte 4 – Observar o desacoplamento

16. **Parar todos os consumers**, mantendo o broker a correr:

```
$ kubectl scale deployment worker --replicas=0
```

17. **Continuar a enviar tarefas** (por script ou pela interface de gestão). Repare que o produtor **funciona normalmente**, sem qualquer erro, apesar de não haver ninguém a consumir.
18. **Observar a acumulação** na interface de gestão (separador *Queues*) ou por linha de comandos:

```
$ kubectl exec deployment/rabbitmq -- rabbitmqctl list_queues name messages
```

Registe o número de mensagens acumuladas.

19. **Voltar a escalar os consumers** e verificar que as mensagens acumuladas são processadas, sem perda:

```
$ kubectl scale deployment worker --replicas=3
$ kubectl logs -f -l app=worker --prefix --max-log-requests=10
```

Esta é a demonstração central do guião: o desacoplamento temporal entre produtor e consumidor.

20. **Testar a durabilidade.** Reinicie o broker e confirme que as mensagens persistentes sobrevivem:

```
$ kubectl scale deployment worker --replicas=0
# enviar algumas mensagens
$ kubectl rollout restart deployment rabbitmq
$ kubectl get pods -l app=rabbitmq --watch
$ kubectl exec deployment/rabbitmq -- rabbitmqctl list_queues name messages
```

Nota: este Deployment não tem armazenamento persistente associado, pelo que o estado é repostado quando o Pod é recriado. Compare o resultado obtido com o que esperaria de um broker implantado com StatefulSet e PersistentVolume, como discutido na teoria.

8 Parte 5 – Redundância e falha de nó

21. **Observar a distribuição dos workers pelos nós:**

```
$ kubectl get pods -l app=worker -o wide
```

22. **Forçar a distribuição por nós diferentes com anti-affinity.** Acrescente ao `spec.template.spec` do `worker.yaml`:

```
affinity:
  podAntiAffinity:
    preferredDuringSchedulingIgnoredDuringExecution:
      - weight: 100
        podAffinityTerm:
          labelSelector:
            matchExpressions:
              - key: app
                operator: In
                values:
                  - worker
          topologyKey: kubernetes.io/hostname
```

```
$ kubectl apply -f worker.yaml
$ kubectl get pods -l app=worker -o wide
```

23. **Simular a falha de um nó** com tarefas em curso. A partir do Proxmox:

```
$ qm stop <vmid-w1>
```

Continue a enviar tarefas e observe se o sistema continua a processá-las nos workers do nó sobrevivente:

```
$ kubectl get pods -l app=worker -o wide
$ kubectl exec deployment/rabbitmq -- rabbitmqctl list_queues name messages
```

24. **Recuperar e limpar:**

```
$ qm start <vmid-w1>
$ kubectl delete -f worker.yaml
$ kubectl delete configmap worker-code
$ kubectl delete -f rabbitmq.yaml
```

9 Entregável / Questões de Verificação

- a) Submeta os ficheiros `rabbitmq.yaml`, `worker.yaml`, `send_task.py`, `worker.py`, `publish_event.py` e `subscribe_event.py`, e uma captura da interface de gestão com mensagens acumuladas na fila.
- b) Qual a diferença de comportamento entre a work queue (Parte 2) e o publish/subscribe (Parte 3) quando existem múltiplos consumers? Explique com base no exchange usado em cada caso.
- c) Descreva o que aconteceu ao producer quando reduziu os workers a zero. Que propriedade da arquitetura permite este comportamento?
- d) Se o desacoplamento via fila não existisse e o producer chamasse o consumer diretamente (comunicação síncrona), o que aconteceria ao producer com o consumer indisponível?
- e) Qual foi o efeito de escalar de 2 para 4 workers no tempo total de processamento? Relacione com a lei de Amdahl (Aula 08): o ganho foi proporcional ao número de workers?
- f) O `worker.py` usa `basic_ack` explícito. O que aconteceria a uma mensagem se o worker morresse a meio do processamento, antes do ACK? Que garantia de entrega é esta?
- g) As mensagens sobreviveram ao reinício do broker? Explique o resultado obtido e indique que alteração ao manifesto seria necessária para as preservar.
- h) Que efeito teve o `podAntiAffinity` na distribuição dos workers? Porque é que isto é relevante para redundância?
- i) O broker está implantado com uma única réplica. Identifique-o como *single point of failure* e proponha duas formas de o mitigar.